.UBAfiuba
FACULTAD DE INGENIERÍA

Sistemas Distribuidos I
(TA050) Curso Roca

Trabajo Práctico Diseño

13 de octubre de 2026

Martino Nervi 111829
Francisco Serrano 111813

Felipe Fialayre 112202

Índice

1. Introducción **4**
1.1. Datos disponibles . . . 4
1.2. Problema a resolver . . . 4
1.3. Idea general . . . 5

2. Escenarios **5**
2.1. Q1: artículos en inglés hasta 2020 . . . 6
2.2. Q2: biografías hasta 2020 . . . 6
2.3. Q3: palabras compartidas entre inglés y francés . . . 6
2.4. Q4: top 20 de palabras en los abstracts . . . 6
2.5. Q5: artículos desde 2021 por encima del promedio . . . 6
2.6. Resumen . . . 7

3. Vista lógica **7**
3.1. DAG . . . 7

4. Vista de desarrollo **8**
4.1. Diagrama de paquetes . . . 8
4.1.1. client . . . 9
4.1.2. gateway . . . 9
4.1.3. worker . . . 9
4.1.4. Common . . . 9

5. Vista física **9**
5.1. Diagrama de robustez . . . 9
5.1.1. Entrada: cliente y gateway . . . 11
5.1.2. Procesamiento y Filtrado: El Componente Filter . . . 12
5.1.3. Q2: Image Downloader . . . 12
5.1.4. Stopwords . . . 12
5.1.5. Q3: Join . . . 12
5.1.6. Q4: Accumulator, Reducer y Sorter . . . 13
5.1.7. Q5: Join Average y Above Avg Filter . . . 13
5.1.8. Salida . . . 14
5.1.9. Finalización . . . 14
5.1.10. Reparto entre réplicas . . . 14
5.2. Diagrama de despliegue . . . 14

6. Vista de procesos **16**
6.1. Diagrama de secuencia . . . 16
6.2. Diagramas de actividad . . . 17
6.2.1. Cliente, gateway . . . 17

6.2.2. Q2 y Q3 19
6.2.3. Q4 y Q5 20

7. Tareas y división **21**

1. Introducción

1.1. Datos disponibles

El sistema trabaja sobre un dataset de Kaggle con artículos de Wikipedia en inglés y en francés, distribuido en archivos en formato parquet. Cada fila corresponde a un artículo completo, y cada archivo contiene del orden de 25.000 artículos organizados en *row groups*, lo que permite leerlo por partes. Las imágenes no están incluidas en el dataset: solo se dispone de su URL, por lo que se obtienen de la API de Wikimedia.

De las columnas disponibles, el sistema utiliza las siguientes:

Columna	Contenido	Uso
`url`	texto	Q1, Q2, Q5 y reparto entre réplicas
`name`	título del artículo	Q2
`in_language`	`en` o `fr`	Q1, Q3
`date_modified`	fecha	corte anterior / posterior a 2021
`abstract`	texto plano	Q3, Q4, Q5
`sections`	JSON con las secciones	Q2
`image.content_url`	URL de la imagen	Q2

Cuadro 1: Columnas del dataset utilizadas

El campo `date_created` se encuentra vacío en casi todos los registros, por lo que la fecha de un artículo se toma de `date_modified`.

Para orientar el diseño analizamos el volumen de cada columna sobre un archivo de muestra (25.000 artículos en inglés):

Columna	Bytes por fila	% del total	Uso
`sections`	∼5.150	92 %	Q2
`abstract`	∼315	6 %	Q3, Q4, Q5
`url`	∼60	1 %	todas
`name`	∼28	<1 %	Q2
`date_modified`	∼19	<1 %	todas
`image.content_url`	∼16	<1 %	Q2
`in_language`	2	<1 %	Q1, Q3

Cuadro 2: Volumen por columna en la muestra

`sections` concentra casi todo el volumen porque contiene el texto completo del artículo, aunque Q2 solo necesita los nombres de las secciones. Además, en la muestra solo el 4,4 % de los artículos es anterior a 2021. Estos datos justifican filtrar por fecha antes de transportar `sections` (ver Sección 5). Los abstracts, en cambio, son cortos (mediana de 38 palabras y máximo de 1.206), por lo que no tiene sentido dividir un artículo para procesarlo.

1.2. Problema a resolver

Se requiere un sistema distribuido que resuelva cinco consultas sobre los artículos de cada cliente, descriptas en detalle en la Sección 2:

1. URL y fecha de modificación de los artículos en inglés hasta 2020.
2. Nombre, URL e imagen de los artículos hasta 2020 con una sección “biography” o “biographie”.

3. Palabras compartidas entre artículos en inglés y en francés.
4. Las 20 palabras más usadas en los abstracts.
5. Artículos desde 2021 con más palabras en el abstract que el promedio de los artículos hasta 2020.

Además, el sistema debe cumplir los siguientes requerimientos no funcionales:

- Atender consultas aisladas de múltiples clientes, con una única ejecución por cliente.
- Estar optimizado para entornos multicomputadora y escalar agregando nodos, tanto respecto de la cantidad de clientes como del volumen de datos de cada uno.
- Contar con un middleware que abstraiga la comunicación basada en grupos.
- Garantizar un cierre ordenado (*graceful quit*) ante la señal `SIGTERM`.

En esta etapa no se contempla la tolerancia a fallos, y la topología del sistema (cantidad de réplicas de cada etapa) se define mediante un archivo de configuración, sin descubrimiento dinámico de nodos. El diseño, de todas formas, busca no impedir su incorporación futura.

1.3. Idea general

Cada cliente se conecta a uno de los gateways del sistema, elegido a partir de su identificador, y le envía su dataset. El gateway realiza solo trabajo liviano: separa cada artículo en tres flujos (Q1, Q2 y abstracts), conserva las columnas que necesita cada uno y aplica los filtros de fecha e idioma. Los flujos se publican en el middleware, a través del cual se comunican todos los nodos de procesamiento.

El procesamiento se organiza en etapas, cada una con varias réplicas. La clave con la que se reparten los datos cambia según lo que necesita cada etapa:

- **Por artículo** (`url`): las etapas que procesan cada artículo de forma independiente, como el tokenizado de abstracts, se reparten de modo que los datos de un mismo cliente se distribuyan entre todas las réplicas.
- **Por palabra** (`client_id, palabra`): las etapas que agregan por palabra (Q3 y Q4) reagrupan los datos para que todas las apariciones de una palabra de un cliente lleguen a la misma réplica.
- **Por cliente** (`client_id`): las etapas que arman el resultado final de un cliente reúnen solo los resultados parciales, ya reducidos.

Los resultados vuelven al gateway que mantiene la conexión con el cliente, que se los envía a medida que llegan. El fin de los datos de cada cliente se propaga por las mismas colas que los datos, de modo que cada etapa sabe cuándo terminó un cliente sin necesidad de un canal de control aparte.

2. Escenarios

El sistema tiene un único caso de uso principal: un cliente envía su conjunto de artículos de Wikipedia (en inglés y francés) y recibe el resultado de cinco consultas. Cada cliente se atiende de forma aislada: sus datos y resultados nunca se mezclan con los de otros clientes que estén siendo procesados al mismo tiempo. A continuación se describe cada consulta, junto con los supuestos que tomamos donde el enunciado deja lugar a interpretación.

En todas las consultas, la fecha de un artículo se toma de `date_modified`: el campo `date_created` se encuentra vacío en la gran mayoría de los registros.

2.1. Q1: artículos en inglés hasta 2020

Se obtienen la URL y la fecha de modificación de todos los artículos escritos en inglés con fecha de modificación anterior al 1 de enero de 2021.

Salida: `(url, date_modified)` por cada artículo.

2.2. Q2: biografías hasta 2020

Se obtienen el nombre, la URL y la imagen de los artículos, en inglés o en francés, con fecha de modificación anterior al 1 de enero de 2021 que tengan entre sus secciones una llamada "biography" o "biographie". El dataset solo contiene la URL de la imagen, por lo que el sistema la descarga desde la API de Wikimedia.

Salida: `(name, url, image_bytes)` por cada artículo.

Supuestos: la comparación del nombre de la sección no distingue mayúsculas de minúsculas. Los artículos sin imagen se informan con la imagen vacía.

2.3. Q3: palabras compartidas entre inglés y francés

Se obtiene el listado de palabras que aparecen en el abstract de al menos un artículo en inglés y también en el de al menos un artículo en francés. No hay corte por fecha.

Salida: listado de palabras, sin repetidos.

Supuestos: se aplica el mismo tratamiento de palabras que en Q4 (minúsculas y sin stopwords), y solo se consideran las palabras de los abstracts.

2.4. Q4: top 20 de palabras en los abstracts

Se obtienen las 20 palabras que aparecen en más abstracts, considerando todos los artículos de ambos idiomas y sin corte por fecha. Dentro de un mismo abstract cada palabra se cuenta una sola vez, y las palabras triviales (stopwords) se excluyen.

Salida: 20 pares `(palabra, cantidad)`.

2.5. Q5: artículos desde 2021 por encima del promedio

Se obtienen la URL y la cantidad de palabras del abstract de los artículos con fecha de modificación desde el 1 de enero de 2021 cuya cantidad de palabras supere el promedio de los artículos anteriores a esa fecha. La cantidad de palabras de un abstract se calcula con el mismo criterio que Q4: palabras distintas y sin stopwords.

Salida: `(url, cantidad_palabras)` por cada artículo.

Supuestos: los artículos anteriores a 2021 sin abstract entran en el promedio con cero palabras.

2.6. Resumen

Consulta	Idioma	Fecha	Salida
Q1	EN	< 2021	url, fecha
Q2	EN + FR	< 2021	nombre, url, imagen
Q3	EN + FR	–	palabras compartidas
Q4	EN + FR	–	top 20 (palabra, cantidad)
Q5	EN + FR	< 2021 vs. ≥ 2021	url, cantidad de palabras

Cuadro 3: Resumen de las consultas

3. Vista lógica

3.1. DAG

El DAG describe qué transformaciones sufren los datos desde que el cliente los envía hasta que se obtiene cada resultado, sin considerar cómo se distribuye el procesamiento. Cada nodo es una operación y cada arista indica qué datos pasan a la siguiente. Las operaciones que comparten varias consultas se realizan una sola vez y el flujo se bifurca recién cuando las consultas necesitan algo distinto.

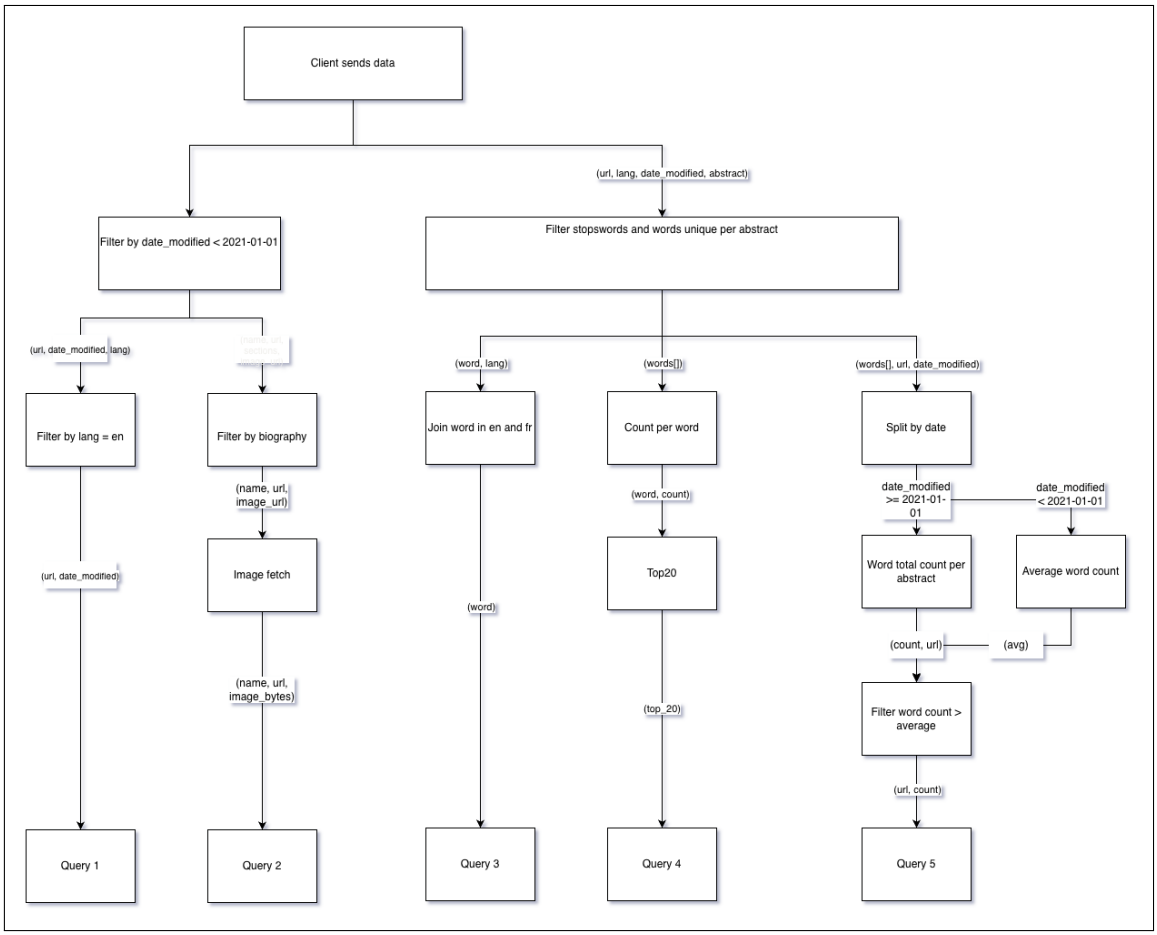


Figura 1: DAG de las cinco consultas

Desde los datos del cliente salen dos ramas principales:

- **Filtro por fecha (Q1 y Q2).** Ambas consultas trabajan solo con artículos anteriores a 2021, por lo que el filtro se aplica una única vez.
  - **Q1** conserva los artículos en inglés y devuelve `(url, date_modified)`.
  - **Q2** conserva los artículos con una sección "biography" o "biographie", descarga su imagen y devuelve `(name, url, image_bytes)`.
- **Tokenizado de abstracts (Q3, Q4 y Q5).** Cada abstract se pasa a minúsculas, se eliminan las stopwords y se conservan las palabras distintas. Las tres consultas parten de este resultado.
  - **Q3** junta las palabras de artículos en inglés con las de artículos en francés y devuelve las que aparecen en ambos idiomas.
  - **Q4** cuenta, para cada palabra, en cuántos abstracts aparece, y devuelve las 20 más frecuentes. Como el tokenizado ya eliminó las repeticiones dentro de cada abstract, cada palabra se cuenta una sola vez por artículo.
  - **Q5** separa los artículos por fecha. Con los anteriores a 2021 calcula la cantidad promedio de palabras por abstract; de los posteriores obtiene la cantidad de palabras de cada uno. Finalmente conserva los artículos posteriores que superan el promedio y devuelve `(url, cantidad)`.

Lógicamente, Q4 y Q5 son independientes. En la implementación (ver Sección 5.1.7) el promedio de Q5 se obtiene del mismo conteo que usa Q4, pero esa es una optimización y no una dependencia del problema.

4. Vista de desarrollo

4.1. Diagrama de paquetes

El diagrama de paquetes muestra cómo se organiza el código fuente y qué módulos dependen de cuáles. El sistema se compone de tres ejecutables (`client`, `gateway` y `worker`) y de un paquete común (`Common`) con el código que comparten.

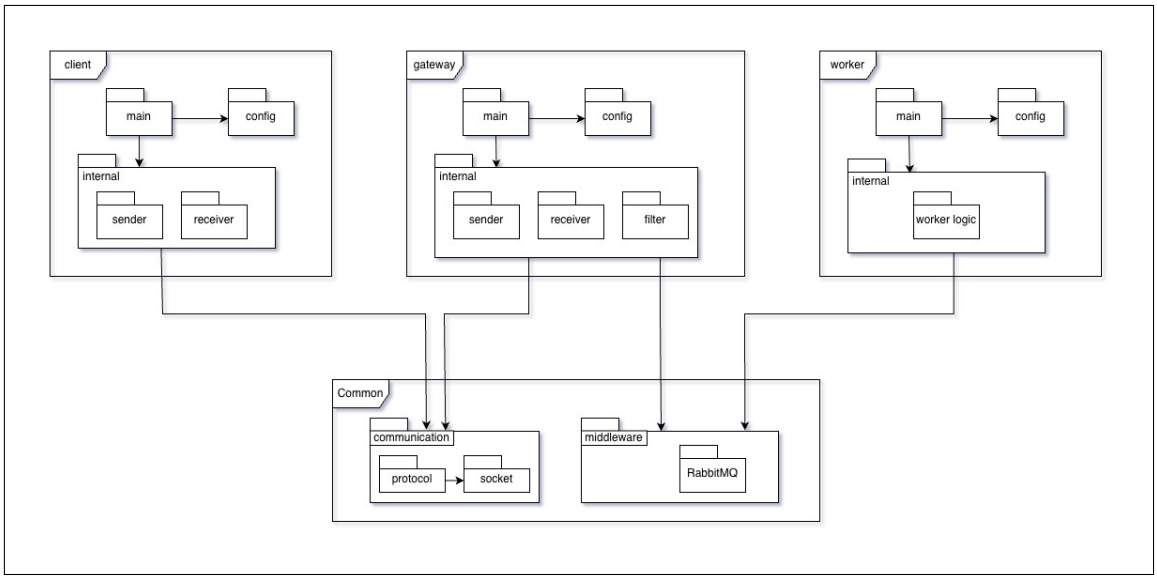


Figura 2: Diagrama de paquetes

Los tres ejecutables siguen la misma estructura:

- `main`: punto de entrada. Lee la configuración, crea los componentes que correspondan y los pone en marcha. También registra el manejo de `SIGTERM` para el cierre ordenado.
- `config`: lectura de los parámetros propios de cada ejecutable, por ejemplo la lista de gateways en el cliente, la cantidad de réplicas de cada etapa en el gateway, o el rol y el número de réplica en el worker.
- `internal`: la lógica propia del ejecutable, que ningún otro paquete utiliza.

4.1.1. client

`sender` lee el dataset y lo envía al gateway en lotes; `receiver` recibe los resultados de cada consulta. Ambos usan el paquete `communication`.

4.1.2. gateway

El gateway tiene dos lados: uno hacia el cliente y otro hacia los workers.

- `receiver` recibe los lotes de cada cliente a través de `communication`.
- `filter` separa cada artículo en los flujos de Q1, Q2 y abstracts, aplicando los filtros de costo constante, y los publica hacia los workers a través del `middleware`.
- `sender` consume, a través del `middleware`, los resultados que llegan a la cola de este gateway, lleva la cuenta de qué consultas terminó cada cliente, y le envía a cada uno sus resultados y el aviso de fin a través de `communication`.

4.1.3. worker

Los distintos nodos de procesamiento (Image Downloader, Stopwords, Filter, Join, Accumulator, Reducer, Sorter, Join Average y Above Avg Filter) se modelan como un único paquete para mantener el diagrama legible. Cada uno tiene su propio punto de entrada y su propia lógica sobre los datos, pero todos comparten la misma estructura y las mismas dependencias: se comunican exclusivamente a través del `middleware`. Image Downloader es el único con una dependencia adicional: consulta la API de Wikimedia por HTTP para descargar las imágenes.

4.1.4. Common

- `communication`: comunicación entre cliente y gateway. `protocol` define el formato de los mensajes (lotes de datos, resultados y fin) y `socket` los envía y recibe por TCP, verificando que se lean y escriban todos los bytes.
- `middleware`: comunicación entre el gateway y los workers, y entre workers. Abstrae la comunicación basada en grupos, de modo que los nodos envían y reciben mensajes de un grupo de réplicas sin conocer los detalles de RabbitMQ, que solo se utiliza dentro de este paquete.

5. Vista física

5.1. Diagrama de robustez

El diagrama de robustez muestra los nodos del sistema, qué datos se envían entre ellos y cómo se reparten entre las réplicas de cada etapa. Para mantenerlo legible, se presenta en dos partes: una vista general del sistema, en la que la resolución de Q4 y Q5 aparece como un único bloque, y un detalle de ese bloque.

Usamos la siguiente notación:

- `( )`: campos que viajan en el mensaje.
- `[ ]`: criterio de reparto entre las réplicas del destino (por ejemplo, `[crc32(url)]`).
- `key=gateway_id`: clave de ruteo de los resultados hacia el gateway.
- `si`: condición de filtrado.
- Círculos apilados: nodo replicado. La cantidad de réplicas de cada etapa se define en el archivo de configuración.

Todos los mensajes llevan además el par `(client_id, gateway_id)`, que identifica al cliente dueño de los datos y al gateway que mantiene su conexión. Todos los nodos que mantienen estado (Join, Accumulator, Reducer, Sorter, Join Average y Above Avg Filter) lo separan por `client_id`, de modo que los datos de distintos clientes nunca se mezclan, y lo liberan cuando el cliente termina.

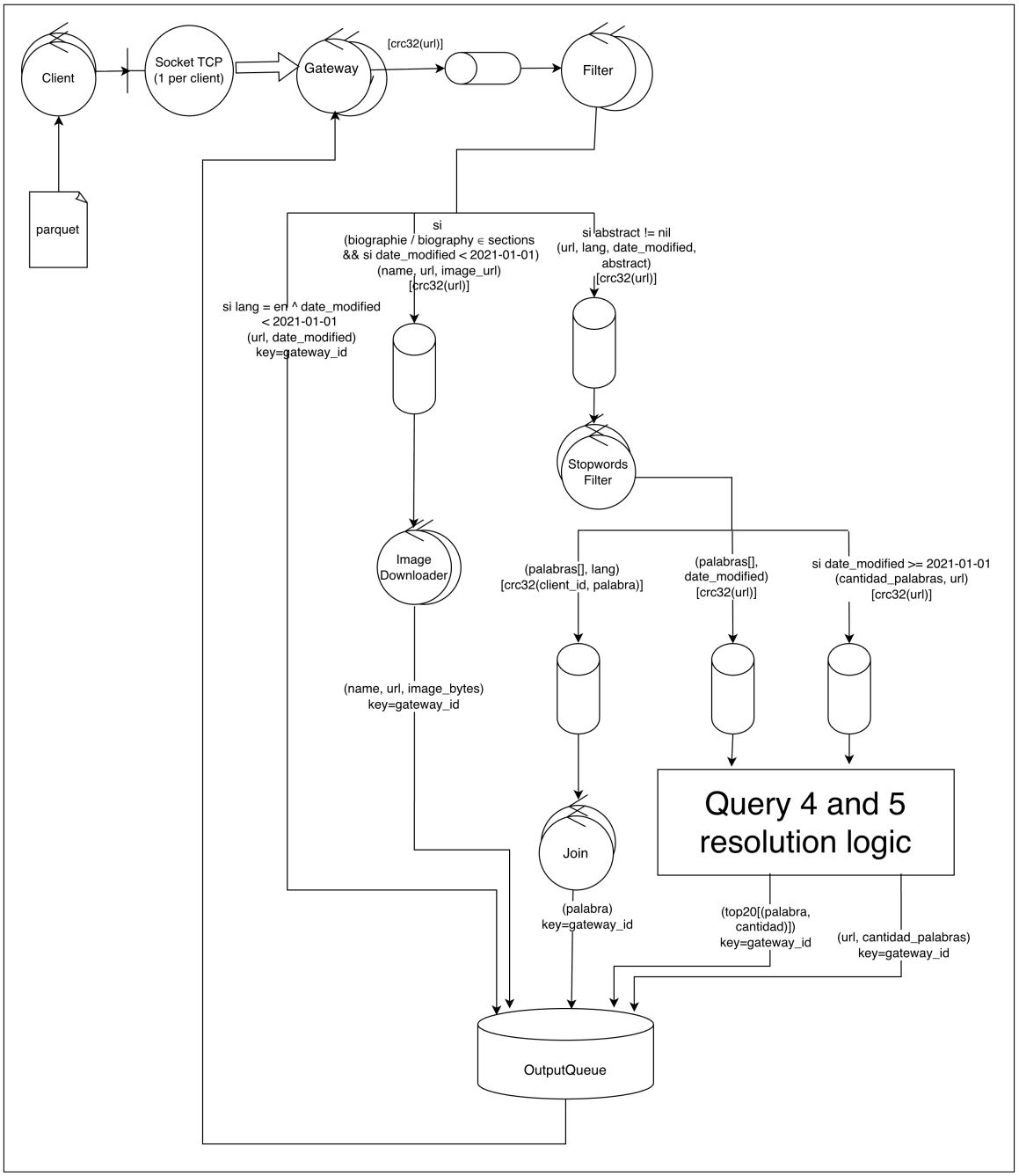


Figura 3: Diagrama de robustez general. La resolución de Q4 y Q5 se detalla en la Figura 4.

5.1.1. Entrada: cliente y gateway

Cada cliente genera un identificador único (`client_id`) y elige un gateway de la lista de la configuración con `crc32(client_id) mod N`; si no responde, prueba con el siguiente. Abre una única conexión TCP, por la que envía el dataset en lotes leídos y luego recibe los resultados.

El gateway solo realiza trabajo de costo constante por fila: **proyecta** cada artículo a las columnas que necesitan las consultas y **publica** el resultado a un único exchange (`records`), repartido entre las réplicas del Filter por `crc32(url)`. No aplica ningún filtro de consulta: el filtrado por fecha e idioma se delega al Filter, que está replicado. La proyección se apoya en el análisis de volumen: la columna `sections` ocupa cerca del 92 % de los datos, pero de ella solo se necesitan los nombres de las secciones (para detectar las biografías de Q2); el gateway descarta el resto antes

de publicar, evitando transportar el texto completo de las secciones. La decisión de no filtrar en el gateway es deliberada: como cada cliente está atado a un único gateway, el trabajo que se hace allí no se reparte entre réplicas, por lo que debe ser mínimo; mover el filtrado al Filter permite que ese costo escale con sus réplicas.

5.1.2. Procesamiento y Filtrado: El Componente Filter

El componente Filter se encarga de recibir los registros publicados por el gateway y aplicar las restricciones correspondientes a las consultas. Al estar replicado, el costo operacional del filtrado por fecha e idioma se distribuye equitativamente entre las distintas instancias utilizando una estrategia de hashing basada en `crc32(url)`. De esta manera, se garantiza que el sistema mantenga una alta escalabilidad horizontal ante el incremento en el volumen de datos entrantes.

5.1.3. Q2: Image Downloader

El Image Downloader recibe las biografías ya filtradas por el Filter `(name, url, image_url)` y se ocupa únicamente de descargar la imagen desde su URL, usando la API de Wikimedia. Envía `(name, url, image_bytes)` a la salida. Como la descarga es la operación dominante (I/O de red), el nodo escala en dos ejes: por réplicas y por paralelismo interno de descargas.

5.1.4. Stopwords

El Stopwords recibe los abstracts del Filter, también repartidos por `crc32(url)`. Es el nodo que resuelve el trabajo por palabra: tokeniza cada abstract (lo pasa a minúsculas, descarta números y stopwords, y conserva las palabras distintas). Es el último nodo que ve el abstract completo; a partir de él solo circulan palabras. Con el resultado abre tres flujos:

- envía `(palabras[], lang)` al Join (Q3);
- envía `(palabras[], date_modified)` a los Accumulators (Q4 y promedio de Q5);
- si el artículo es de 2021 en adelante, envía `(url, cantidad_palabras)` al Above Avg Filter (Q5).

5.1.5. Q3: Join

El Join guarda, por cliente y por palabra, si apareció en un artículo en inglés y en uno en francés, y emite la palabra en cuanto se cumplen ambas condiciones. Se reparte por `crc32(client_id, palabra)`: así todas las apariciones de una palabra de un mismo cliente llegan a la misma réplica, sin importar de qué artículo provengan.

5.1.6. Q4: Accumulator, Reducer y Sorter

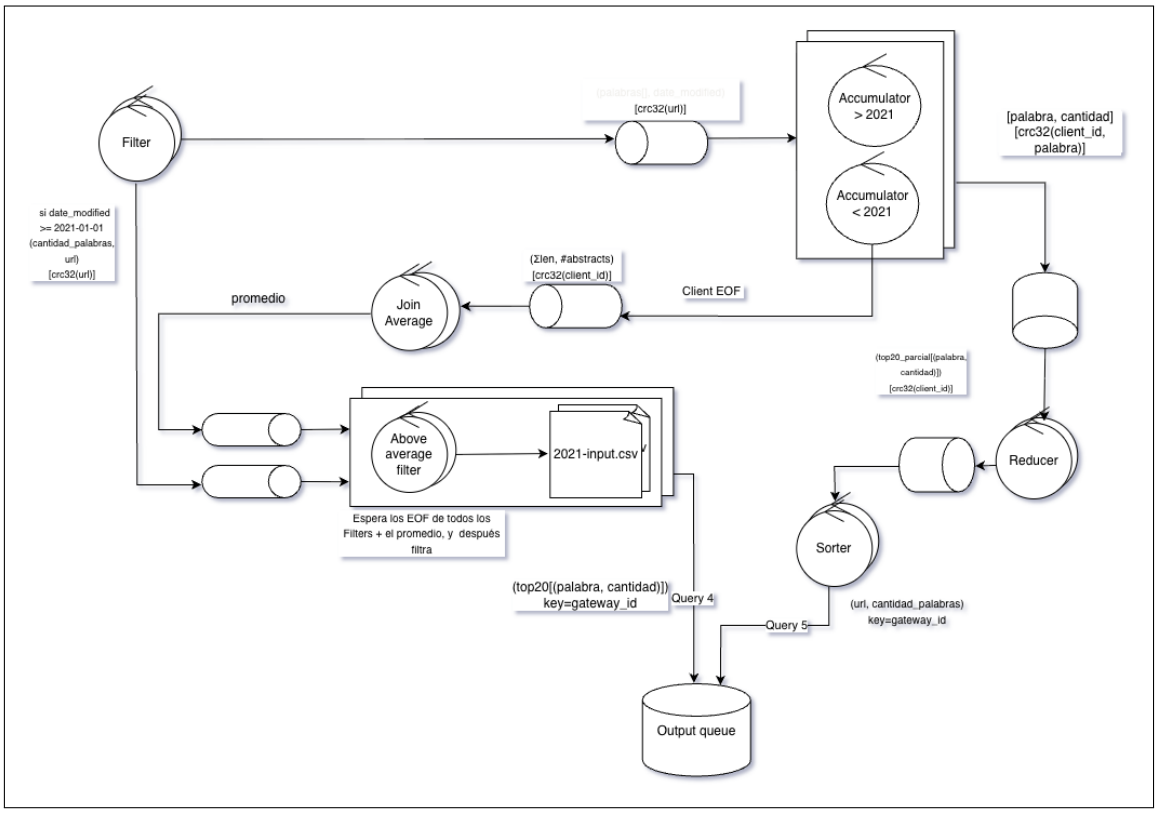


Figura 4: Detalle de la resolución de Q4 y Q5

Cada Accumulator recibe artículos repartidos por URL y lleva, por palabra, la cantidad de abstracts en los que aparece. Este conteo se mantiene por separado para los artículos anteriores a 2021 y para los posteriores. Q4 no distingue fechas, pero la separación permite reutilizar el conteo de los anteriores a 2021 para calcular el promedio que necesita Q5, sin recorrer los datos una segunda vez (ver Sección 5.1.7).

Al recibir el EOF de todos los Filters, el Accumulator suma ambos conteos y reparte el resultado por `crc32(client_id, palabra)` entre los Reducers. Cada Reducer obtiene así el conteo total de sus palabras y calcula un top 20 parcial. El Sorter recibe los tops parciales del cliente, uno por Reducer, y arma el top 20 final.

El top 20 parcial alcanza: una palabra que está en el top 20 global también está en el top 20 de su Reducer, porque a lo sumo 19 de las palabras de ese Reducer pueden superarla. Sin esta etapa, el Sorter debería recibir el vocabulario completo de cada Accumulator y sumarlo solo, lo que no escalaría con el volumen de datos de un cliente.

5.1.7. Q5: Join Average y Above Avg Filter

Al terminar el cliente, el Accumulator de fechas anteriores a 2021 envía al Join Average la suma de palabras y la cantidad de abstracts de su parte. Join Average junta los parciales de todos los Accumulators, calcula el promedio y lo envía a todas las réplicas de Above Avg Filter.

Above Avg Filter se reparte por URL: cada réplica guarda en disco su parte de los artículos desde 2021 del cliente. Funciona como una barrera: recién cuando recibió el fin de datos (EOF) de cada uno de los Filters (uno por réplica, ya que los artículos del cliente se reparten entre todos ellos) y el promedio, recorre lo guardado y emite los artículos que lo superan (el mecanismo de finalización se describe en la Sección 5.1.9). Repartir por URL permite que el trabajo de un cliente

con mucho volumen se distribuya entre varias réplicas. Cada réplica es independiente: en cuanto tiene su parte de los datos y el promedio, emite sus resultados y su EOF, sin coordinarse con las demás.

Lógicamente, Q4 y Q5 son independientes. En la implementación, el promedio se obtiene del mismo Accumulator que cuenta palabras para Q4: la suma de las cantidades por palabra es igual a la suma de las cantidades de palabras por abstract.

5.1.8. Salida

Cada consulta publica sus resultados con el `gateway_id` como clave de ruteo, de modo que llegan a la cola del gateway que tiene la conexión con el cliente. Se usa el `gateway_id` y no un hash del `client_id` porque el cliente pudo haberse conectado a otro gateway si el primero no respondía. Q1, Q2 y Q3 se envían a medida que se producen; Q4 y Q5 se envían al final.

5.1.9. Finalización

Cuando el cliente termina de enviar, el gateway envía un mensaje de fin (EOF) a todas las réplicas del Filter, por la misma cola (`records`) que los datos. Como la cola respeta el orden, el EOF siempre llega después de los datos. El Filter lo propaga a los nodos que alimenta (Image Downloader y Stopwords), y así cada etapa lo pasa a la siguiente; cada nodo da por terminado a un cliente cuando recibió un EOF de cada uno de sus productores. El gateway, a su vez, espera el fin de cada consulta contando un EOF por réplica productora: `FilterAmount` para Q1, uno por Image Downloader para Q2, uno por Join para Q3, uno por Above Avg Filter para Q5, y uno solo (del Sorter) para Q4. Cuando completó las cinco, se lo informa al cliente y cierra la conexión.

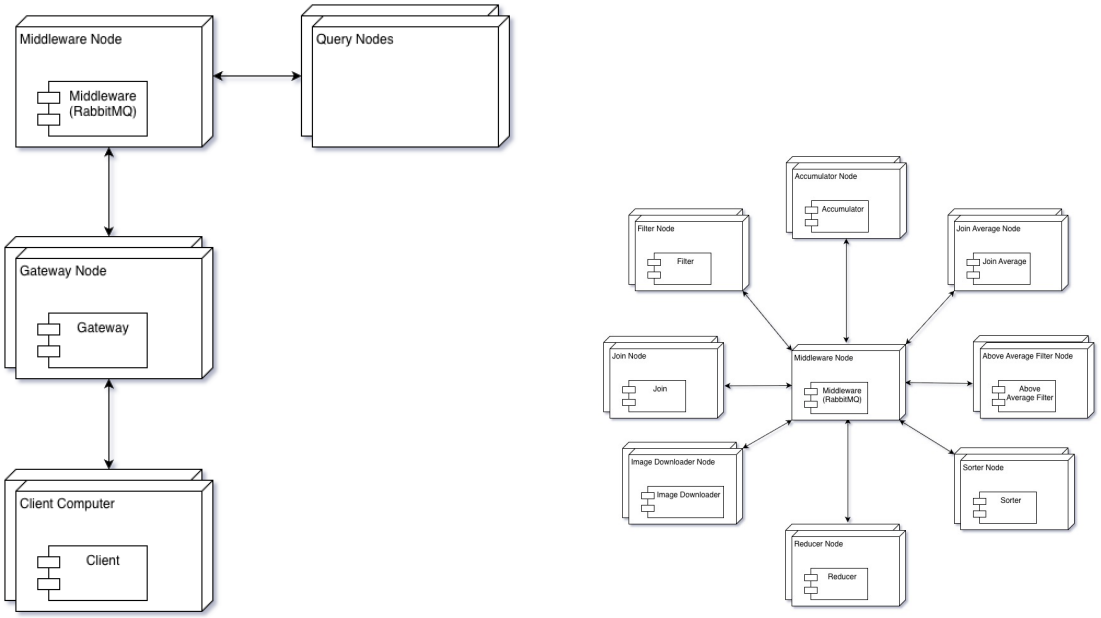
5.1.10. Reparto entre réplicas

Etapa	Clave de reparto	Motivo
Gateway	`crc32(client_id)`	repartir clientes
Image Downloader, Filter, Stopwords, Accumulator, Above Avg Filter	`crc32(url)`	repartir artículos
Join, Reducer	`crc32(client_id, palabra)`	totales por palabra
Sorter, Join Average	`crc32(client_id)`	resultado por cliente
Salida	`gateway_id`	volver al gateway del cliente

Cuadro 4: Criterio de reparto de cada etapa

5.2. Diagrama de despliegue

El diagrama de despliegue muestra en qué nodos se ejecuta cada componente y cómo se conectan. Se presenta en dos partes: a la izquierda, una vista general en la que los nodos que resuelven las consultas se agrupan como *Query Nodes*; a la derecha, el detalle de esos nodos.



(a) Vista externa

(b) Vista interna

Figura 5: Diagrama de despliegue

Cada componente corre en su propio contenedor Docker. Los nodos dibujados apilados se replican, y la cantidad de réplicas de cada tipo se configura de forma independiente. El cliente se conecta por TCP a un gateway. El gateway y todos los nodos de procesamiento se comunican exclusivamente a través del nodo de middleware (RabbitMQ): ningún nodo se conecta directamente con otro, lo que permite agregar o quitar réplicas sin modificar a los demás. La única conexión hacia fuera del sistema es la del Image Downloader con la API de Wikimedia, por HTTPS, para descargar las imágenes.

6. Vista de procesos

6.1. Diagrama de secuencia

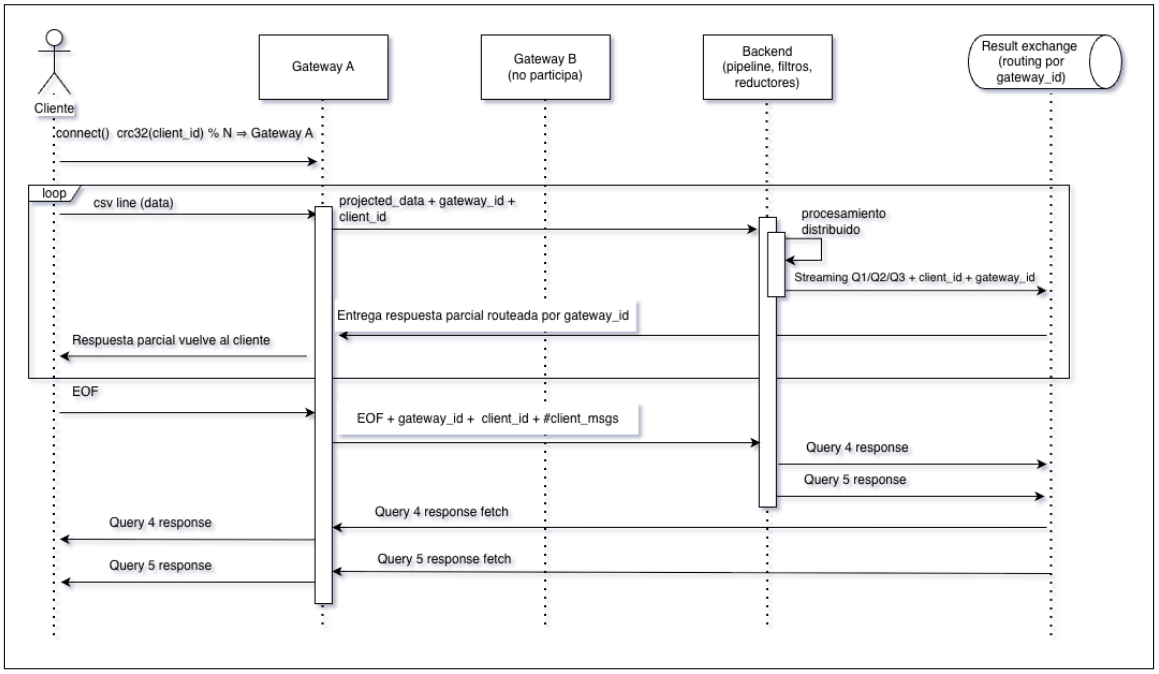


Figura 6: Secuencia de finalización de un cliente

El diagrama de secuencia muestra el ciclo de vida completo de un pedido, desde la conexión del cliente hasta la entrega de las cinco respuestas. El cliente determina su gateway mediante crc32(client_id) mód N (el Gateway A en este escenario) y abre una única conexión TCP que conserva durante toda la sesión; los restantes gateways (Gateway B) no intervienen.

A continuación, envía su conjunto de datos en lotes: por cada lote recibido, el Gateway A proyecta las columnas útiles y las publica hacia los nodos de procesamiento (el Backend en el diagrama), estampando en cada mensaje el par (client_id, gateway_id). El client_id permite aislar el estado de cada cliente en los nodos con memoria, mientras que el gateway_id asegura que los resultados retornen a su origen. Dado que las consultas Q1, Q2 y Q3 se resuelven en modalidad de *streaming*, sus respuestas parciales se generan a medida que el Backend procesa el flujo de datos, sin esperar a la finalización de la ingesta, y se envían de vuelta al cliente por el camino de retorno.

Una vez finalizado el envío de datos, el cliente emite una señal de EOF que el Gateway A propaga hacia los nodos de procesamiento. A partir de dicha notificación, y tras haber recibido cada etapa el fin de flujo de la totalidad de sus productores, se procesan Q4 y Q5; dado que ambas consultas dependen del procesamiento global de los *abstracts*, sus resultados se emiten al final.

El camino de retorno constituye el núcleo del diseño: toda respuesta, sea parcial o final, se publica en el *exchange* de resultados utilizando el gateway_id como clave de ruteo (*routing key*). De esta forma, el mensaje se deposita exclusivamente en la cola del gateway que mantiene la sesión TCP activa con dicho cliente, garantizando que el flujo regrese al nodo de origen para su posterior reenvío. Se opta por rutear mediante gateway_id en lugar de recalcular crc32(client_id) debido a que el cliente podría haberse conectado a una réplica alternativa ante una falta de respuesta inicial; estampar el gateway_id durante la ingesta desacopla la topología y asegura el direccionamiento exacto hacia el propietario real de la conexión TCP.

6.2. Diagramas de actividad

Los diagramas de actividad describen el comportamiento de cada nodo: qué hace al recibir un mensaje, cómo distingue datos de fin de datos (EOF) y cuándo emite sus resultados. Se presentan siguiendo el recorrido de los datos, desde el cliente hasta la salida. Los nodos replicados (marcados con $\times N$) ejecutan la misma lógica en cada réplica.

6.2.1. Cliente, gateway

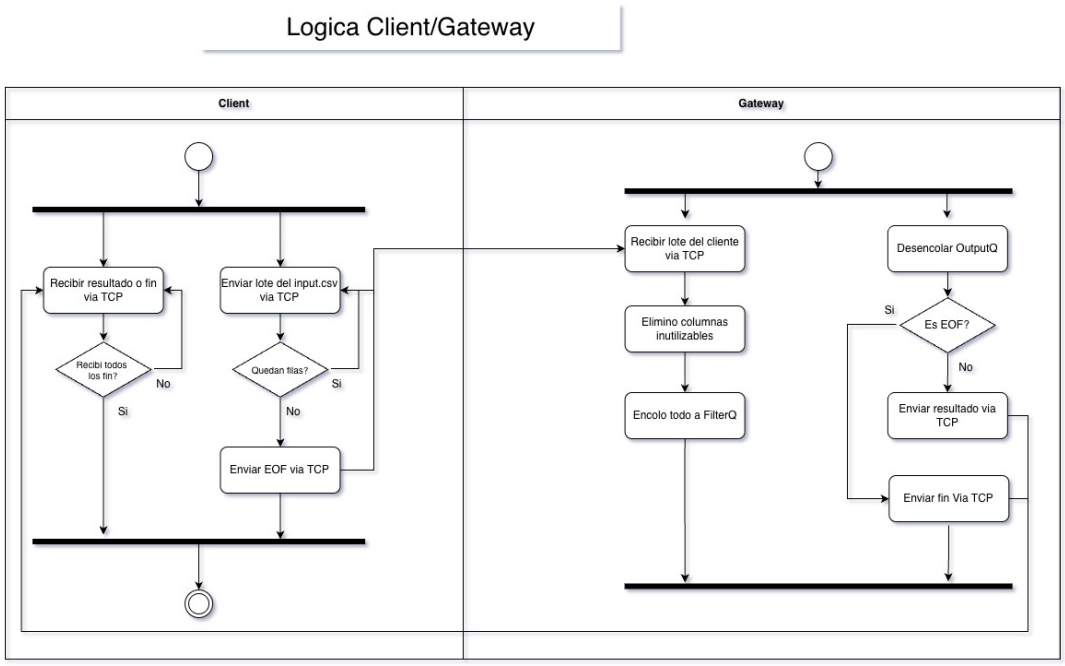


Figura 7: Actividades del cliente y del gateway

El cliente realiza dos tareas en paralelo. Por un lado, lee el dataset y envía lotes al gateway por TCP hasta que no quedan filas, y entonces envía el EOF. Por otro, recibe los resultados de las consultas a medida que llegan, hasta haber recibido el fin de las cinco.

El gateway trabaja en paralelo. Mientras recibe los lotes del cliente, distribuye cada artículo entre las réplicas del Filter, eligiendo la réplica según un hash de la URL, de modo que un mismo artículo siempre llega al mismo destino. Al recibir el EOF del cliente, lo propaga a todas las réplicas del Filter. En paralelo, consume su cola de salida: reenvía al cliente los resultados de cada consulta, agrupados en lotes, y al recibir el fin de una consulta, que llega una vez por cada réplica del nodo que la resuelve, se lo informa al cliente.

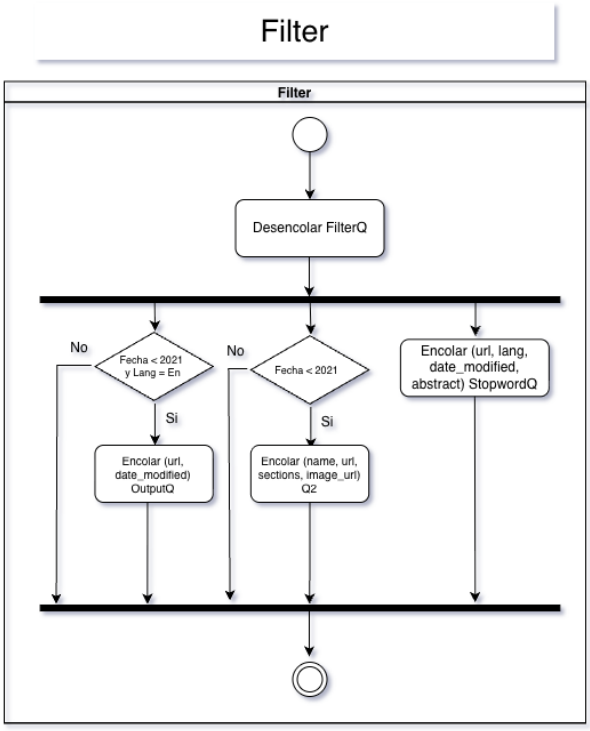


Figura 8: Ruteo de un lote en el gateway

El Filter evalúa de forma independiente los tres flujos para cada artículo. Si es anterior a 2021 y está en inglés, envía (url, date modified) directamente a la salida (Q1). Si es anterior a 2021 y contiene una sección de biografía, envía (name, url, image url) al Image Downloader (Q2). Si tiene abstract, envía (url, lang, date modified, abstract) al Stopwords Filter, que resuelve Q3, Q4 y Q5. Al recibir el EOF, lo propaga a todos sus destinos y envía a la salida el fin de Q1.

6.2.2. Q2 y Q3

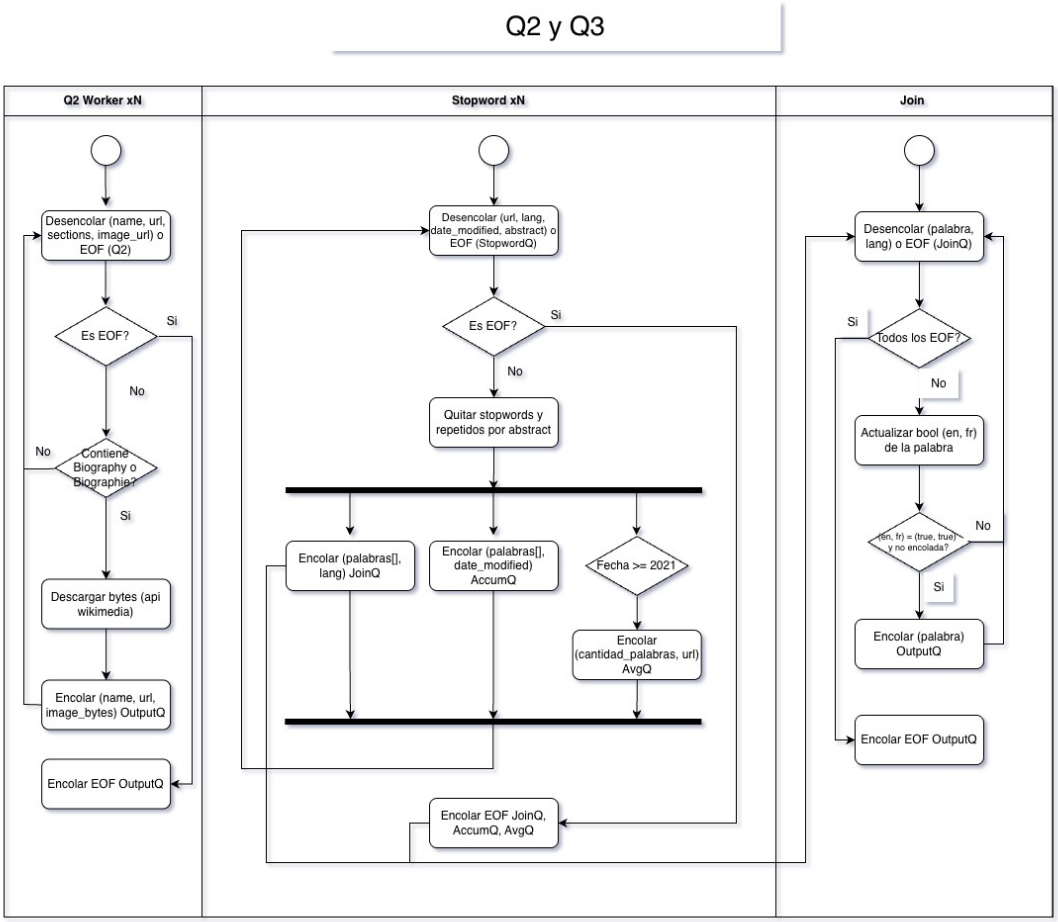


Figura 9: Actividades de Image Downloader, Filter y Join

El Image Downloader procesa cada artículo de forma independiente: si contiene una sección "biography" o "biographie", descarga la imagen de la API de Wikimedia y envía el resultado a la salida. Al recibir el EOF, lo propaga a la salida.

El Stopwords quita las stopwords y las palabras repetidas de cada abstract, y luego envía en paralelo las palabras con su idioma al Join, las palabras con la fecha a los Accumulators y, si el artículo es de 2021 en adelante, su cantidad de palabras al Above Average Filter. Al recibir el EOF, lo propaga a las tres colas.

El Join actualiza, para cada palabra, si apareció en inglés y en francés. Cuando se cumplen ambas condiciones por primera vez, emite la palabra. Una vez recibido el EOF de todos los Stopwords, envía el fin de Q3 a la salida.

6.2.3. Q4 y Q5

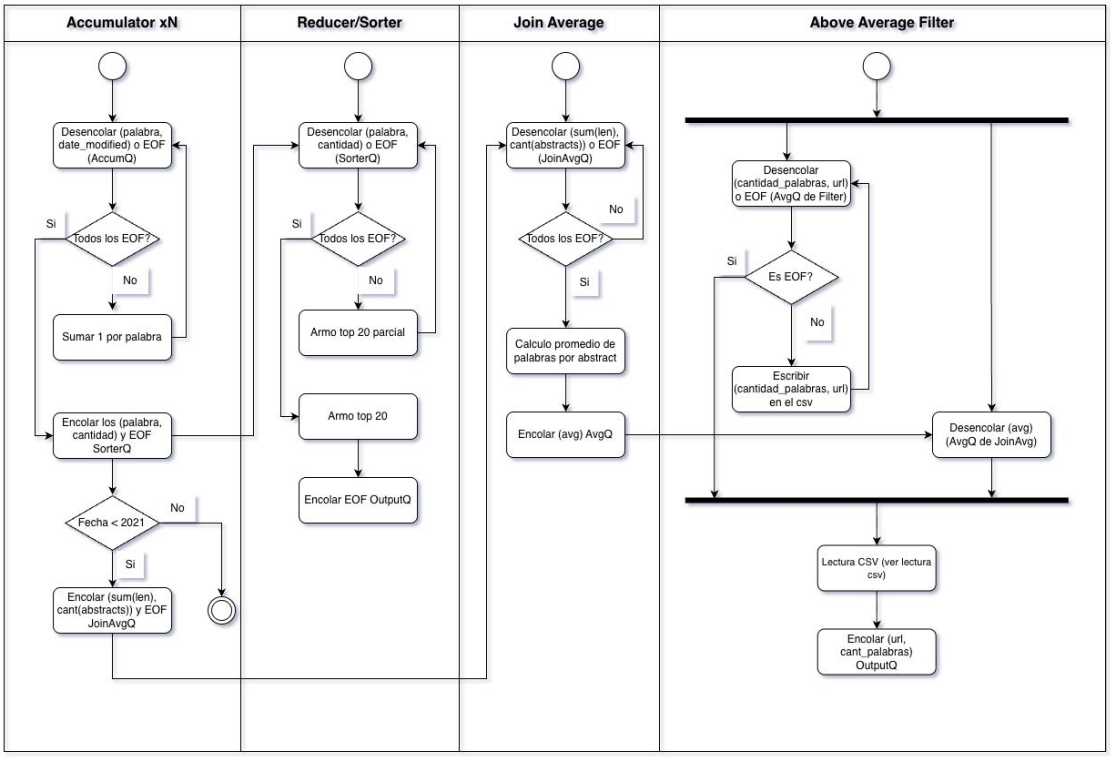


Figura 10: Actividades de Accumulator, Reducer/Sorter, Join Average y Above Average Filter

El Accumulator suma uno por cada palabra recibida. Cuando recibió el EOF de todos los Filters, envía los pares `(palabra, cantidad)` a los Reducers y, para los artículos anteriores a 2021, la suma de palabras y la cantidad de abstracts al Join Average.

Cada Reducer, al recibir todos sus EOF, arma el top 20 parcial de sus palabras; el Sorter combina los tops parciales en el top 20 final y lo envía a la salida junto con el fin de Q4.

El Join Average espera los parciales de todos los Accumulators, calcula el promedio de palabras por abstract y lo envía al Above Average Filter.

El Above Average Filter realiza dos tareas en paralelo: guarda en un archivo los artículos de 2021 en adelante que recibe de los Filters, hasta recibir todos sus EOF, y espera el promedio. Cuando completó ambas, recorre el archivo (Figura 11) y envía a la salida los artículos que superan el promedio.

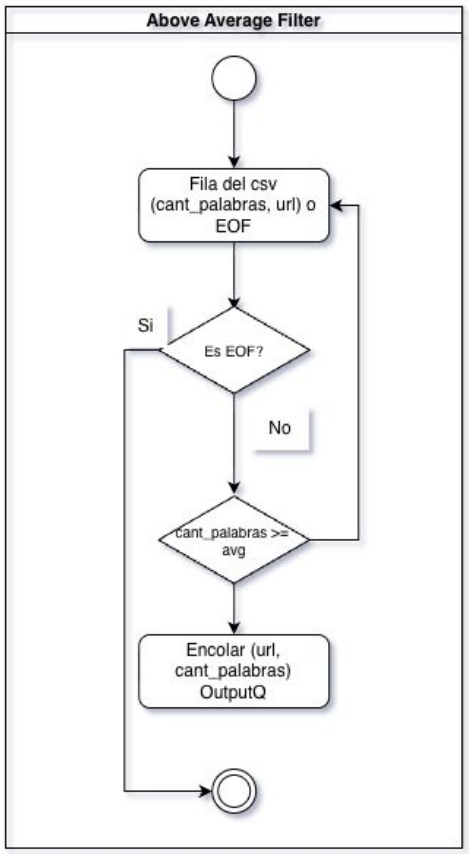


Figura 11: Lectura del archivo en el Above Average Filter

La lectura recorre el archivo fila por fila y envía a la salida cada artículo cuya cantidad de palabras supera el promedio, hasta llegar al final del archivo.

7. Tareas y división

Tarea	Responsable
Middleware (envío a una réplica y a todo el grupo)	Martino
Worker base (recepción, conteo de EOF, SIGTERM)	Martino
Filter y Join (Q3)	Martino
Docker Compose y configuración	Martino
Protocolo cliente–gateway	Felipe
Cliente (lectura de parquet)	Felipe
Gateway (sesiones, ruteo y salida)	Felipe
Q1 y Q2 Worker	Felipe
Accumulator, Reducer y Sorter (Q4)	Francisco
Join Average y Above Avg Filter (Q5)	Francisco
Informe	Francisco

Cuadro 5: Tareas y división entre integrantes